

Action Event Listener en JSF



Action Event Listener en JSF

Concepto de Action Event Listener

En JSF, un **Action Event Listener** es un mecanismo que permite escuchar y manejar eventos que ocurren cuando un usuario interactúa con componentes como botones o enlaces. Estos eventos generalmente representan acciones específicas realizadas por el usuario, como hacer clic en un botón para enviar un formulario o redirigir a otra página.

El **Action Event Listener** es ideal para ejecutar lógica personalizada cuando ocurre algunas de las acciones mencionadas.

Diferencia entre Action y ActionListener

- **action:** Define una acción que regresa un resultado, como una navegación (por ejemplo, redirigir a otra página).
- **actionListener:** Es un listener que escucha el evento pero no afecta la navegación; se utiliza para lógica adicional antes o después de la acción.

Flujo del Action Event Listener

1. El usuario hace clic en un botón o enlace.

2. Se genera un evento que es interceptado por el **Action Listener**.
3. El listener ejecuta una lógica específica antes de continuar con las demás fases del ciclo de vida de JSF.

Sintaxis Básica

Configuración en el archivo XHTML

Un componente como un botón o enlace puede utilizar el atributo `actionListener` para definir el método del backing bean que manejará el evento.

```
<h:commandButton value="Enviar" actionListener="#{miBean.procesarEvento}" />
```

Atributos importantes:

- **actionListener:** Apunta al método del backing bean que manejará el evento.
- **action:** (Opcional) Puede utilizarse junto con el listener para definir la navegación.

Implementación en el Backing Bean

El método asociado al `actionListener` debe aceptar un parámetro de tipo `ActionEvent`. Este objeto contiene información sobre el componente que disparó el evento.

```
import jakarta.faces.event.ActionEvent;

public void procesarEvento(ActionEvent event) {
    // Lógica para manejar el evento
    System.out.println("Evento procesado: " + event.getComponent().getId());
}
```

Proyecto Completo

Paso 1: Configuración inicial del proyecto

1. **Crear el proyecto:**
 - En NetBeans, selecciona **File > New Project**.
 - Escoge **Java with Maven > Web Application** y presiona **Next**.
 - Configura el proyecto:
 - **Project Name:** `ActionEventJSF`
 - Selecciona **Jakarta EE** como la versión de la plataforma.
 - Configura GlassFish como servidor.

Paso 2: Crear los componentes

1. Bean de modelo: Candidato.java: Este bean almacena los datos del candidato.

Descripción:

- Almacena información como nombre, apellido, salario deseado, entre otros.
- Incluye logs para seguimiento en consola.

```
package beans.model;

import jakarta.enterprise.context.SessionScoped;
import jakarta.inject.Named;
import java.io.Serializable;
import java.util.Date;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@Named("candidato")
@SessionScoped
public class Candidato implements Serializable {

    private String nombre;
    private String apellido;
    private int salarioDeseado;
    private Date fechaNacimiento;
    private String codigoPostal;
    private String colonia;
    private String ciudad;
    private String comentario;

    private final Logger log = LoggerFactory.getLogger(Candidato.class);

    public Candidato() {
        log.info("Creando el objeto Candidato");
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
        log.info("Modificando la propiedad de nombre:" + this.nombre);
    }

    public String getApellido() {
```

```
        return apellido;
    }

    public void setApellido(String apellido) {
        this.apellido = apellido;
        log.info("Modificando la propiedad de apellido:" + this.apellido);
    }

    public int getSalarioDeseado() {
        return salarioDeseado;
    }

    public void setSalarioDeseado(int salarioDeseado) {
        this.salarioDeseado = salarioDeseado;
        log.info("Modificando la propiedad de sueldoDeseado" + this.salarioDeseado);
    }

    public Date getFechaNacimiento() {
        return fechaNacimiento;
    }

    public void setFechaNacimiento(Date fechaNacimiento) {
        this.fechaNacimiento = fechaNacimiento;
    }

    public String getCodigoPostal() {
        return codigoPostal;
    }

    public void setCodigoPostal(String codigoPostal) {
        this.codigoPostal = codigoPostal;
    }

    public String getColonia() {
        return colonia;
    }

    public void setColonia(String colonia) {
        this.colonia = colonia;
    }

    public String getCiudad() {
        return ciudad;
    }
}
```

```
public void setCiudad(String ciudad) {
    this.ciudad = ciudad;
}

public String getComentario() {
    return comentario;
}

public void setComentario(String comentario) {
    this.comentario = comentario;
}
}
```

2. Backing Bean: VacanteForm.java: Este bean maneja la lógica de la aplicación y procesa eventos como la validación del código postal.

Descripción:

- Define acciones como `enviar()` y listeners para cambios en los campos del formulario.
- Utiliza inyección de dependencias para interactuar con el bean `Candidato`.

```
package beans.backing;

import beans.model.Candidato;
import jakarta.enterprise.context.RequestScoped;
import jakarta.faces.application.FacesMessage;
import jakarta.faces.component.UIInput;
import jakarta.faces.component.UIViewRoot;
import jakarta.faces.context.FacesContext;
import jakarta.faces.event.ActionEvent;
import jakarta.faces.event.ValueChangeEvent;
import jakarta.inject.Inject;
import jakarta.inject.Named;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@Named
@RequestScoped
public class VacanteForm extends Object {

    @Inject
    private Candidato candidato;

    private boolean comentarioEnviado;
```

```
private final Logger log = LoggerFactory.getLogger(Candidato.class);

public VacanteForm() {
    log.info("Creando el objeto VacanteForm");
}

public void setCandidato(Candidato candidato) {
    this.candidato = candidato;
}

public String enviar() {
    if ("Juan".equals(candidato.getNombre()) &&
        "Perez".equals(candidato.getApellido())) {
        String msg = "Gracias Juan Perez, te has registrado correctamente.";
        FacesMessage facesMessage =
            new FacesMessage(FacesMessage.SEVERITY_ERROR, msg, null);
        FacesContext.getCurrentInstance().addMessage(null, facesMessage);
        log.info("Usuario registrado correctamente");
        return "index"; // Redirige a la misma página
    }
    log.info("Datos de usuario inválidos");
    return "fallo"; // Redirige a una página de éxito
}

public void codigoPostalListener(ValueChangeEvent valueChangeEvent) {
    FacesContext facesContext = FacesContext.getCurrentInstance();
    UIViewRoot uiViewRoot = facesContext.getViewRoot();
    String nuevoCodigoPostal =
        (String) valueChangeEvent.getNewValue();
    if ("03810".equals(nuevoCodigoPostal)) {
        UIInput coloniaInputText =
            (UIInput) uiViewRoot.findComponent("vacanteForm:colonia");
        String nuevaColonia = "Napoles";
        coloniaInputText.setValue(nuevaColonia);
        coloniaInputText.setSubmittedValue(nuevaColonia);

        UIInput ciudadInputText =
            (UIInput) uiViewRoot.findComponent("vacanteForm:ciudad");
        String nuevaCiudad = "Ciudad de Mexico";
        ciudadInputText.setValue(nuevaCiudad);
        ciudadInputText.setSubmittedValue(nuevaCiudad);

        facesContext.renderResponse();
    }
}
```

```

    }

    public void ocultarComentario(ActionEvent actionEvent) {
        this.comentarioEnviado = !this.comentarioEnviado;
    }

    public boolean isComentarioEnviado() {
        return comentarioEnviado;
    }

    public void setComentarioEnviado(boolean comentarioEnviado) {
        this.comentarioEnviado = comentarioEnviado;
    }
}

```

3. Vista JSF: index.xhtml: Esta vista permite al usuario ingresar y validar información del candidato.

Descripción:

- Incluye un formulario para capturar información.
- Implementa validaciones en campos como salario y fecha de nacimiento.

```

<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:h="http://java.sun.com/jsf/html"
      xmlns:f="http://xmlns.jcp.org/jsf/core">
<head>
    <title>Hola Mundo JSF</title>
</head>
<body>
    <h:form id="vacanteForm">
        <h:messages globalOnly="true" ></h:messages>

        <table>
            <tr>
                <td><h:outputLabel for="nombre"
                    value="#{msgs['vacanteForm.nombre']}"></h:outputLabel></td>
                <td><h:inputText id="nombre" required="true"
                    value="#{candidato.nombre}"></h:inputText></td>
                <td><h:message for="nombre"></h:message></td>
            </tr>
            <tr>
                <td><h:outputLabel for="apellido"
                    value="#{msgs['vacanteForm.apellido']}"></h:outputLabel></td>

```

```

        <td><h:inputText id="apellido" required="true"
            value="#{candidato.apellido}"></h:inputText></td>
        <td><h:message for="apellido"></h:message></td>
    </tr>
    <tr>
        <td><h:outputLabel for="salarioDeseado"
            value="#{msgs['vacanteForm.salarioDeseado']}"></td>
        <td><h:inputText id="salarioDeseado" required="true"
            value="#{candidato.salarioDeseado}"
            <f:validateLongRange minimum="500" maximum="5000"/>
            </h:inputText>
        </td>
        <td><h:message for="salarioDeseado"></h:message></td>
    </tr>
    <tr>
        <td><h:outputLabel for="fechaNacimiento"
            value="#{msgs['vacanteForm.fechaNacimiento']}"></h:outputLabel></td>
        <td><h:inputText id="fechaNacimiento" required="true"
            value="#{candidato.fechaNacimiento}"
            <f:convertDateTime pattern="dd/MM/yyyy"></f:convertDateTime>
            </h:inputText>
        </td>
        <td><h:message for="fechaNacimiento"></h:message></td>
    </tr>
    <tr>
        <td><h:outputLabel for="codigoPostal"
            value="#{msgs['vacanteForm.codigoPostal']}" ></h:outputLabel></td>
        <td><h:inputText id="codigoPostal" immediate="true"
            onchange="this.form.submit()" required="true"
            value="#{candidato.codigoPostal}"
            valueChangeListener="#{vacanteForm.codigoPostalListener}">
            </h:inputText></td>
        <td><h:message for="codigoPostal"></h:message></td>
    </tr>
    <tr>
        <td><h:outputLabel for="colonia"
            value="#{msgs['vacanteForm.colonia']}" ></h:outputLabel></td>
        <td><h:inputText id="colonia" required="true"
            value="#{candidato.colonia}"></h:inputText></td>
        <td><h:message for="colonia"></h:message></td>
    </tr>
    <tr>
        <td><h:outputLabel for="ciudad" value="#{msgs['vacanteForm.ciudad']}"
            ></h:outputLabel></td>
        <td><h:inputText id="ciudad" required="true"

```



```

        value="#{candidato.ciudad}"></h:inputText></td>
      <td><h:message for="ciudad"></h:message></td>
    </tr>
  </table>
  <table>
    <tr>
      <td>
        <h:commandLink actionListener="#{vacanteForm.ocultarComentario}"
          immediate="true"
          rendered="#{!vacanteForm.comentarioEnviado}"
          value="#{msgs['vacanteForm.mostrar']}" />
        <h:commandLink actionListener="#{vacanteForm.ocultarComentario}"
          immediate="true"
          rendered="#{vacanteForm.comentarioEnviado}"
          value="#{msgs['vacanteForm.ocultar']}" />
      </td>
    </tr>
  </table>
  <h:panelGroup rendered="#{vacanteForm.comentarioEnviado}">
    <table>
      <tr>
        <td><h:outputLabel for="comentario"
          value="#{msgs['vacanteForm.comentario']}" /></td>
      </tr>
      <tr>
        <td><h:inputTextarea id="comentario" cols="50" rows="10"
          value="#{candidato.comentario}" /></td>
      </tr>
    </table>
  </h:panelGroup>
  <h:commandButton action="#{vacanteForm.enviar}"
    value="#{msgs['vacanteForm.enviar']}" />
</h:form>
</body>
</html>

```

El atributo `immediate="true"` en el código de un componente JSF afecta el flujo del ciclo de vida de JSF, especificando que la acción asociada al componente se debe ejecutar en las primeras fases del ciclo de vida, incluso antes de que se realicen validaciones de entrada o la actualización de valores en el modelo.

Significado de `immediate="true"`: Permite que el `ValueChangeListener` (método `codigoPostalListener` del backing bean) se ejecute inmediatamente después de que el valor del campo cambie, sin esperar a que se completen las fases de validación o actualización del modelo.

Uso práctico: Esto es útil cuando necesitas realizar alguna acción inmediatamente después de un cambio en el valor (por ejemplo, actualizar otros campos del formulario como `colonia` y `ciudad` basados en el código postal) sin depender de la validación de todo el formulario.

Significado de `immediate="true` en el componente de `command link`: Asegura que la acción (`ocultarComentario`) se ejecute sin esperar la validación o la actualización del modelo de todo el formulario.

Uso práctico: Esto permite que el cambio en el estado del comentario (`comentarioEnviado`) ocurra inmediatamente al hacer clic, independientemente de si otros campos del formulario tienen valores válidos o no.

4. Archivos de recursos: `mensajes_es.properties` y `mensajes_en.properties`

Estos archivos almacenan las cadenas de texto para internacionalización.

1. `mensajes_es.properties`

```
vacanteForm.nombre=Nombre
vacanteForm.apellido=Apellido
vacanteForm.salarioDeseado=Salario Deseado
vacanteForm.fechaNacimiento=Fecha de Nacimiento
vacanteForm.codigoPostal=Codigo Postal
vacanteForm.colonia=Colonia
vacanteForm.ciudad=Ciudad
vacanteForm.comentario=Comentario
vacanteForm.mostrar=Mostrar
vacanteForm.ocultar=Ocultar
vacanteForm.enviar=Enviar
```

2. `mensajes_en.properties`

```
vacanteForm.nombre=Name
vacanteForm.apellido=Last Name
vacanteForm.salarioDeseado=Desired Salary
vacanteForm.fechaNacimiento=Birthdate
vacanteForm.codigoPostal=Zip Code
vacanteForm.colonia=Suburb
vacanteForm.ciudad=City
vacanteForm.comentario=Annotation
vacanteForm.mostrar=Show
vacanteForm.ocultar=Hide
vacanteForm.enviar=Submit
```

Paso 3: Configuración de GlassFish y despliegue

1. **Despliega la aplicación:**
 - Haz clic derecho en el proyecto y selecciona **Run**.
 - NetBeans compilará y desplegará la aplicación en GlassFish.
2. **Prueba la aplicación:**
 - Abre un navegador y accede a la aplicación

Diferencias entre Value Change Listener y Action Event Listener en JSF

Ambos mecanismos son utilizados para manejar eventos en JavaServer Faces (JSF), pero tienen propósitos y comportamientos diferentes.

Aspecto	Value Change Listener	Action Event Listener
Propósito	Manejar cambios en los valores de los componentes de entrada.	Manejar eventos generados por acciones del usuario, como hacer clic en un botón o enlace.
Evento Disparador	Se dispara cuando el usuario modifica el valor de un componente de entrada.	Se dispara cuando el usuario realiza una acción (por ejemplo, presionar un botón).
Ciclo de Vida	Se ejecuta durante la fase Apply Request Values , antes de que el modelo sea actualizado.	Se ejecuta durante la fase Invoke Application , después de que el modelo ha sido actualizado.
Método Requerido	Recibe un parámetro de tipo <code>ValueChangeEvent</code> .	Recibe un parámetro de tipo <code>ActionEvent</code> .
Uso Típico	Validar o modificar datos relacionados antes de actualizar el modelo.	Ejecutar lógica que responde directamente a la acción del usuario (como guardar datos o redirigir).
Ejemplo de Componente	Usado típicamente con <code>h:inputText</code> , <code>h:selectOneMenu</code> , etc.	Usado típicamente con <code>h:commandButton</code> , <code>h:commandLink</code> , etc.
Atributo en XHTML	<code>valueChangeListener</code>	<code>actionListener</code>
Orden de Ejecución	Se ejecuta antes de la actualización del modelo de datos.	Se ejecuta después de la actualización del modelo de datos.
Impacto en la Navegación	No afecta directamente la navegación de la aplicación.	Puede usarse junto con el atributo <code>action</code> para navegación.

Ejemplo de Value Change Listener

Vista (XHTML):

```
<h:inputText value="#{miBean.valor}"
             valueChangeListener="#{miBean.manejarCambio}"
             immediate="true" />
```

Backing Bean (Java):

```
public void manejarCambio(ValueChangeEvent event) {  
    String nuevoValor = (String) event.getNewValue();  
    System.out.println("Nuevo valor ingresado: " + nuevoValor);  
}
```

Ejemplo de Action Event Listener

Vista (XHTML):

```
<h:commandButton value="Guardar" actionListener="#{miBean.procesarAccion}" />
```

Backing Bean (Java):

```
public void procesarAccion(ActionEvent event) {  
    System.out.println("Botón presionado: " + event.getComponent().getId());  
}
```

Cuándo Usar Cada Uno

- **Value Change Listener:**
 - Cuando necesitas reaccionar a cambios en los valores de entrada antes de que estos sean aplicados al modelo.
 - Ejemplo: Actualizar campos relacionados con un código postal.
- **Action Event Listener:**
 - Cuando necesitas ejecutar lógica después de una acción específica realizada por el usuario.
 - Ejemplo: Procesar un formulario, guardar datos o redirigir al usuario a otra página.

Ambos mecanismos son complementarios y se pueden usar en conjunto en una misma aplicación según los requerimientos de cada caso.

Saludos!

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)